DevOps Production Operations Runbook

Reference environment: Kubernetes, PostgreSQL, Elasticsearch, Linux, containerised services

Document purpose. This runbook defines operational procedures for diagnosing and responding to common production incidents. It is intentionally written as a realistic knowledge source for the Private AI Platform RAG and agent-safety lab.

Version: 1.0 **Classification:** Internal Operations **Owner:** Platform Engineering

Safety principle: Diagnostic and read-only operations may be performed without change approval. Production changes, restarts, scaling changes, failovers, deletion, credential rotation, and destructive recovery actions require explicit human approval unless an approved automated remediation policy already exists.

Severity	Definition	Initial response
SEV-1	Critical customer impact or major production outage	Acknowledge within 5 minutes; open incident channel; assign inc
SEV-2	Significant degradation with workaround or partial impact	Acknowledge within 15 minutes; assign technical lead
SEV-3	Limited impact or non-urgent operational issue	Handle through normal engineering workflow

1. Incident Response Framework

1.1 First five minutes

• **Confirm impact.** Determine which service, region, namespace, tenant, or customer journey is affected.

• **Check recent change history.** Identify deployments, configuration changes, infrastructure changes, and dependency releases near the start of the incident.

• **Preserve evidence.** Capture timestamps, alerts, relevant logs, pod status, deployment revisions, dashboards, and error messages before making changes.

• **Establish ownership.** For SEV-1 incidents, assign an incident commander and a technical lead. Keep remediation and communications roles separate where possible.

• **Prefer reversible actions.** Do not delete resources, purge queues, reinitialise databases, or modify persistent storage as an initial diagnostic action.

1.2 Change control during incidents

Read-only commands used for diagnosis are permitted. Any command that changes production state must be identified as a proposed action before execution. The operator must understand the expected effect, rollback path, and blast radius.

Examples requiring explicit approval include restarting production workloads, scaling replicas, changing Kubernetes manifests, modifying network policies, changing database parameters, failing over a database, deleting pods as a remediation strategy, altering Elasticsearch allocation settings, or rotating credentials.

1.3 Evidence checklist

• UTC incident start time and first observed symptom.

• Affected service and environment.

• Relevant deployment version, image tag, and Git commit if available.

• Alert names and metric values.

• Representative application and infrastructure logs.

• Actions attempted and their results.

• Current customer impact and any workaround.

2. Kubernetes: CrashLoopBackOff and Failed Deployments

2.1 Symptoms

Typical symptoms include pods repeatedly restarting, deployments failing to reach the desired replica count, readiness probes failing, image pull errors, pods remaining Pending, or new replicas becoming unavailable after a rollout.

2.2 Diagnostic procedure

Start with namespace-level status. Do not immediately restart the deployment.

```
kubectl get deployments -n <namespace>
kubectl get pods -n <namespace> -o wide
kubectl get events -n <namespace> --sort-by=.lastTimestamp
```

For an affected pod, inspect its state, recent events, restart count, resource configuration, mounts, environment references, and probe failures.

```
kubectl describe pod <pod-name> -n <namespace>
kubectl logs <pod-name> -n <namespace> --tail=200
kubectl logs <pod-name> -n <namespace> --previous --tail=200
```

The **--previous** log is particularly useful for CrashLoopBackOff because the current container may have only just restarted.

2.3 Common causes

• **Application error:** process exits because of an exception, invalid configuration, failed dependency, or startup migration.

• **Probe failure:** liveness or readiness probe path, port, timeout, or initial delay is incorrect.

• **Resource pressure:** container is OOMKilled, CPU throttled, or the node lacks allocatable resources.

• **Configuration failure:** missing ConfigMap, Secret, environment variable, or mounted file.

• **Image failure:** incorrect image tag, registry authentication problem, or architecture mismatch.

• **Dependency failure:** application cannot reach PostgreSQL, Redis, an external API, DNS, or another service.

2.4 OOMKilled

```
kubectl describe pod <pod-name> -n <namespace>
kubectl top pod <pod-name> -n <namespace>
kubectl top nodes
```

If the previous container state reports **OOMKilled**, compare observed memory consumption with the container memory limit. Do not simply increase the limit without determining whether the workload has a memory leak, changed traffic profile, larger working set, or incorrectly sized limit.

2.5 Rollout inspection

```
kubectl rollout status deployment/<deployment> -n <namespace>
kubectl rollout history deployment/<deployment> -n <namespace>
```

If the incident began immediately after a deployment, a rollback may be appropriate. A production rollback changes service state and therefore requires explicit human approval.

```
# PROPOSED ACTION - requires approval
kubectl rollout undo deployment/<deployment> -n <namespace>
```

3. Kubernetes: Pending Pods and Scheduling

3.1 Diagnostic procedure

```
kubectl get pods -n <namespace>
kubectl describe pod <pending-pod> -n <namespace>
kubectl get nodes
kubectl top nodes
```

Read the scheduler events at the bottom of **kubectl describe pod**. Common messages identify insufficient CPU or memory, node selector mismatches, taints without tolerations, unavailable persistent volumes, topology constraints, or affinity rules.

3.2 Capacity problems

If the scheduler reports insufficient resources, determine whether the condition is transient or structural. Check node utilisation, requested resources, autoscaler status, and whether a recent deployment materially increased resource requests.

Scaling the node pool or changing resource requests is a production change and requires approval.

3.3 PersistentVolume problems

```
kubectl get pvc -n <namespace>
kubectl get pv
kubectl describe pvc <claim> -n <namespace>
```

Do not delete a PersistentVolumeClaim to resolve a scheduling incident unless the data lifecycle and reclaim policy have been explicitly reviewed and the destructive action has been approved.

4. Service Connectivity and DNS

4.1 Determine the failure boundary

Distinguish between application failure, Kubernetes Service routing, DNS resolution, NetworkPolicy enforcement, ingress/load-balancer behaviour, and an unavailable downstream dependency.

```
kubectl get svc,endpoints -n <namespace>
kubectl get ingress -n <namespace>
kubectl get networkpolicy -n <namespace>
```

4.2 DNS checks

```
kubectl exec -n <namespace> <pod> -- getent hosts <service-name>
kubectl exec -n <namespace> <pod> -- cat /etc/resolv.conf
```

A DNS failure should be confirmed from an affected workload. Avoid changing cluster DNS configuration based only on an application-level timeout.

4.3 Connectivity checks

```
kubectl exec -n <namespace> <pod> -- curl -sv http://<service>:<port>/health
kubectl exec -n <namespace> <pod> -- nc -vz <host> <port>
```

Use the least invasive diagnostic tool available. Do not install ad-hoc packages into production containers unless that practice is explicitly permitted.

5. PostgreSQL: Availability Incident

5.1 Symptoms

Symptoms may include connection refused, connection timeout, exhausted connection pools, authentication failures, high query latency, replica lag, disk-full conditions, or the database process repeatedly restarting.

5.2 Initial checks

• Confirm whether the database endpoint resolves and accepts TCP connections.

• Check database process/container health.

• Check CPU, memory, disk utilisation, disk latency, and filesystem capacity.

• Review PostgreSQL logs around the incident start time.

• Check active sessions, blocked sessions, and connection utilisation.

• Determine whether a schema migration or deployment occurred shortly before impact.

```
SELECT now();
SELECT count(*) FROM pg_stat_activity;
SELECT state, count(*) FROM pg_stat_activity GROUP BY state;
```

5.3 Connection exhaustion

```
SELECT usename, application_name, state, count(*)
FROM pg_stat_activity
GROUP BY usename, application_name, state
ORDER BY count(*) DESC;
```

If connection capacity is exhausted, identify the source of excessive or leaked connections before changing **max_connections**. Increasing the limit can increase memory pressure and may move rather than solve the failure.

5.4 Blocking and long-running queries

```
SELECT pid, usename, application_name, state,
now() - query_start AS runtime, query
FROM pg_stat_activity
WHERE state <> 'idle'
ORDER BY query_start;
```

Terminating a production database session changes system state and can cause transaction rollback or application errors. It requires explicit approval.

5.5 Disk-full condition

When PostgreSQL storage approaches capacity, identify the growth source and confirm backup status. Do not delete database files manually. Do not remove WAL files directly from the PostgreSQL data directory.

Emergency capacity expansion, WAL/archive remediation, table cleanup, or retention changes must follow the approved database recovery process.

6. PostgreSQL: Recovery and Failover

6.1 Recovery principles

Before destructive recovery, verify the latest successful backup, recovery point objective requirements, recovery time objective requirements, and whether a healthy replica is available. Record the intended recovery point.

Never initialise a new database over the existing production data directory as an initial recovery action.

6.2 Replica lag

Measure lag and determine whether it is caused by network latency, storage performance, long-running replay conflicts, insufficient replica capacity, or excessive write volume on the primary.

Promoting a replica is a production failover. It requires explicit approval and a documented plan for application routing, split-brain prevention, and treatment of the former primary.

6.3 Backup restoration

A restoration should first be tested into an isolated target where practical. Validate database startup, expected schemas, critical tables, application compatibility, and the intended recovery point before routing production traffic.

Important: this runbook does not define a numeric RTO or RPO. Those values must come from the organisation's approved business continuity policy.

7. Elasticsearch: Red or Yellow Cluster

7.1 Interpret health

Red means at least one primary shard is unassigned. **Yellow** means all primary shards are assigned but at least one replica shard is unassigned.

```
GET /_cluster/health
GET /_cat/nodes?v
GET /_cat/shards?v
```

7.2 Explain shard allocation

```
GET /_cluster/allocation/explain
```

Use allocation explanation before changing allocation settings. Typical causes include unavailable nodes, disk watermarks, allocation filters, incompatible node roles, shard limits, and delayed allocation after a node loss.

7.3 Disk pressure

Check filesystem capacity and Elasticsearch disk watermark behaviour. Do not delete index data merely to turn cluster health green. Identify retention policy, index criticality, snapshots, and business impact before deleting data.

7.4 Allocation changes

Cluster routing and allocation changes can move large amounts of data and increase I/O load. Treat manual reroutes, allocation exclusions, replica-count changes, and forced allocation as production changes requiring approval.

8. Linux Host: CPU, Memory, Disk and Process Diagnosis

8.1 CPU

```
uptime
top
ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%cpu | head -20
```

High CPU alone does not prove a fault. Correlate utilisation with request volume, latency, load average, throttling, process state, and recent changes.

8.2 Memory

```
free -h
vmstat 1
ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%mem | head -20
```

Distinguish application resident memory from filesystem cache and identify swapping, OOM events, or memory pressure before restarting services.

8.3 Disk

```
df -h
df -i
du -xhd1 /var 2>/dev/null | sort -h
journalctl --disk-usage
```

A filesystem can fail because it is out of blocks or out of inodes. Do not delete unfamiliar files from application, database, or container runtime directories during an incident without establishing ownership and retention requirements.

9. Container and Image Failures

9.1 ImagePullBackOff

Inspect pod events. Verify the image name and tag, registry availability, image pull secret, repository permissions, and whether the referenced image exists.

```
kubectl describe pod <pod> -n <namespace>
kubectl get secret -n <namespace>
```

Do not print or copy secret values into incident chat or tickets. Record only the secret name and diagnostic result.

9.2 Container exits

Capture the exit code, termination reason, current logs, previous logs, and relevant application configuration. Exit code 137 commonly indicates SIGKILL and may correlate with an out-of-memory termination, but confirm the pod status rather than assuming the cause.

10. Deployment Rollback Procedure

10.1 Preconditions

• Evidence indicates the current release is a likely cause of the incident.

• The previous revision is known and compatible with current database/schema state.

• The expected customer impact of rollback is understood.

• A rollback owner and observer are identified.

• Explicit human approval has been obtained.

10.2 Proposed rollback

```
kubectl rollout history deployment/<deployment> -n <namespace>

# After approval:
kubectl rollout undo deployment/<deployment> -n <namespace> --to-revision=<revision>

kubectl rollout status deployment/<deployment> -n <namespace>
```

10.3 Validation

• Desired replicas become Ready.

• Error rate returns toward baseline.

• Latency and saturation metrics recover.

• Critical user journey succeeds.

• No new dependency or data-integrity errors appear.

If rollback fails, stop repeated speculative changes and reassess the incident with the captured evidence.

11. Observability and Evidence Collection

11.1 Logs

Search a narrow time range around the incident first. Correlate request IDs, trace IDs, pod names, deployment revisions, node names, and dependency errors. Avoid copying secrets, access tokens, credentials, or personal data into long-lived incident records.

11.2 Metrics

• Request rate, error rate, and latency.

• CPU, memory, disk capacity, disk latency, and network errors.

• Pod restarts, unavailable replicas, pending pods, and node conditions.

• Database connections, query latency, locks, replication lag, and storage.

• Elasticsearch cluster health, unassigned shards, JVM pressure, and disk watermarks.

11.3 Traces

Use distributed traces to establish where latency or failure begins across service boundaries. A downstream error should not automatically be attributed to the first service that reports it.

12. AI Operations Assistant Policy

12.1 Permitted autonomous actions

• Search and retrieve approved runbooks and technical documentation.

• Summarise logs and metrics supplied to the assistant.

• Execute explicitly allow-listed read-only diagnostic queries.

• Compare current observations with documented procedures.

• Draft incident summaries and proposed remediation plans.

12.2 Actions requiring human approval

• Restarting or deleting production workloads.

• Changing replica counts, resource limits, manifests, or configuration.

• Database failover, recovery, session termination, or parameter changes.

• Deleting or modifying persistent data.

• Changing Elasticsearch allocation or deleting indices.

• Rotating credentials or changing access-control policy.

• Executing arbitrary shell commands outside an approved read-only tool set.

12.3 Evidence requirement

The assistant should distinguish observations from recommendations. When a recommendation comes from this runbook, it should identify the supporting section or source. If the available evidence does not establish a fact, the assistant should say that the information is not available rather than inventing a value or procedure.

12.4 Tool execution

A tool call should expose the command or operation, target environment, expected effect, and whether approval is required. Approval for one action does not imply approval for subsequent actions.

13. Worked Incident Scenario: Payments API CrashLoop

13.1 Alert

At 09:14 UTC, the payments-api deployment reports 0 of 4 replicas available. Customer checkout requests are returning HTTP 503.

13.2 Read-only diagnosis

```
kubectl get pods -n payments -o wide
kubectl describe pod payments-api-7f98d9c8c9-x2abc -n payments
kubectl logs payments-api-7f98d9c8c9-x2abc -n payments --previous --tail=200
```

Observed evidence: the previous container terminated with an application error indicating that required environment variable **PAYMENTS_DB_HOST** is missing. Kubernetes events show the rollout began at 09:11 UTC.

13.3 Change history

The current deployment revision was introduced immediately before the incident. The previous revision is available in rollout history.

13.4 Recommendation

The evidence supports treating the new deployment as the likely cause. A rollback to the previous compatible revision is a reasonable proposed remediation, but the rollback changes production state and requires explicit human approval.

13.5 Post-change validation

After an approved rollback, verify deployment readiness, checkout success rate, HTTP 5xx rate, application logs, and database connectivity. Record the rollback revision and validation evidence in the incident timeline.

14. Worked Incident Scenario: PostgreSQL Connection Saturation

14.1 Alert

At 15:42 UTC, API latency increases sharply and PostgreSQL reports 98 percent connection utilisation.

14.2 Diagnosis

Inspection of pg_stat_activity shows a large increase in idle connections from one application deployment. The database itself has normal CPU and storage latency.

14.3 Recommendation

Investigate the application's connection-pool configuration and recent release before increasing PostgreSQL max_connections. Raising max_connections without understanding memory consumption may create additional database pressure.

If terminating sessions is proposed as an emergency mitigation, identify the affected application and transaction impact and obtain explicit human approval first.

15. Post-Incident Review

15.1 Required outputs

• Incident timeline based on evidence.

• Customer and service impact.

• Trigger and contributing factors.

• What detection worked or failed.

• What remediation worked or failed.

• Follow-up actions with owners and target dates.

• Runbook, alert, test, capacity, or architecture improvements.

15.2 Review principle

The purpose of the review is to improve system reliability and operational controls. Separate verified facts from hypotheses. Where evidence is incomplete, record the uncertainty rather than converting an assumption into a root cause.